Guía práctica integrada: patrón DAO con archivos JSON, de la consola a una aplicación móvil

Facultad de Ingeniería y Sistemas · Oct 13, 2026 · @Benjamin

Grupo 3

1. Propósito y competencias

Esta guía une en una sola ruta las prácticas 2 y 3: primero se aprende el patrón **Data Access Object (DAO)** con un **archivo JSON** en un programa de consola (Parte I) y después se lleva ese mismo patrón a una **aplicación móvil Android** hecha en equipo (Parte II). La idea central no cambia en ningún momento: quien usa el DAO no sabe dónde se guardan los datos; hoy es un archivo `.json`, mañana puede ser MySQL o SQLite, sin modificar el resto de la aplicación.

Asignatura: Desarrollo de Aplicaciones Multiplataforma (DAP) · **Duración sugerida:** 6 horas de clase (2 h la Parte I y 4 h la Parte II, en 2 sesiones) · **Lenguaje:** Java · **Modalidad:** individual en la Parte I y equipos de 3 estudiantes en la Parte II · **Plataformas:** Java 17+ con Maven (Parte I) y Android (Parte II)

Competencias

- **Saber:** explica qué es el patrón DAO, qué papel cumplen la entidad, la interfaz, la implementación, una clase base genérica y una fábrica, y por qué separar el acceso a datos facilita la reutilización de código y permite que varias personas trabajen en paralelo.
- **Saber hacer:** implementa un CRUD completo (crear, listar, buscar, actualizar y eliminar) con un DAO que persiste en un archivo JSON, primero en consola y luego en una aplicación móvil con dos entidades relacionadas cuyos datos viven en el almacenamiento interno del dispositivo.
- **Saber ser:** es diligente al aplicar patrones de diseño que reutilizan código: nombra con claridad, separa responsabilidades, cumple su rol en el equipo, integra su trabajo con el de los demás y deja el proyecto ordenado para quien lo mantenga.

2. Requisitos previos

La Parte I solo necesita las herramientas de Java; la Parte II añade las de Android y las de trabajo en equipo.

Recurso	Detalle	Se usa en
JDK	Versión 17 o superior	Parte I
IDE	NetBeans, IntelliJ IDEA o VS Code con extensión de Java	Parte I
Gestor de dependencias	Maven	Parte I
Android Studio	Versión estable reciente	Parte II

Recurso	Detalle	Se usa en
	(incluye el JDK y el emulador)	
Proyecto Android	Plantilla *Empty Views Activity*, lenguaje Java, paquete `sv.edu.ufg.agenda`, minSdk 24	Parte II
Dispositivo de prueba	Emulador (AVD) o celular con depuración USB	Parte II
Librería JSON	Gson (se agrega en `pom.xml` en la Parte I y en Gradle en la Parte II)	Ambas
Librería de listas	RecyclerView (se agrega en Gradle)	Parte II
Control de versiones	Git con un repositorio por equipo (GitHub o GitLab)	Parte II
Conocimientos	Clases, interfaces, colecciones (`List`) y manejo de excepciones; para la Parte II, además, Activities y layouts XML	Ambas

3. Ruta de la guía: qué cambia de la Parte I a la Parte II

El patrón es el mismo en las dos partes; lo que crece es la complejidad propia de un proyecto real. La Parte I deja el DAO funcionando con una entidad. La Parte II lo **generaliza** (clase base y fábrica), lo **conecta** con pantallas táctiles y lo **organiza** para que tres personas trabajen a la vez.

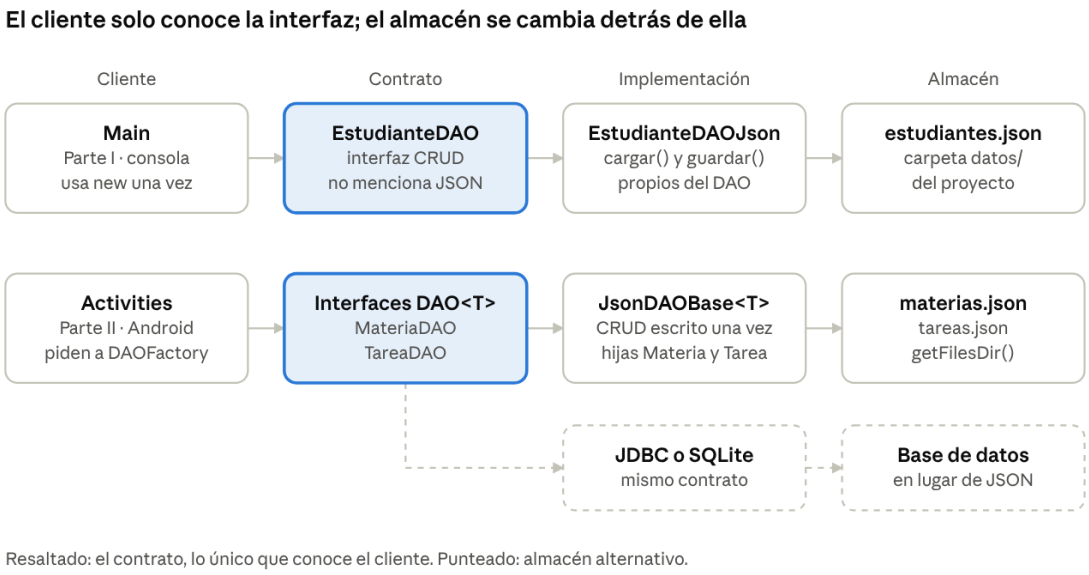
Aspecto	Parte I: consola	Parte II: aplicación móvil
Plataforma	Programa de consola en Java con Maven	Aplicación Android con pantallas táctiles
Ubicación del JSON	Carpeta `datos/` del proyecto	Almacenamiento interno de la app (`getFilesDir()`)
Entidades	Una (`Estudiante`)	Dos relacionadas: `Materia` y `Tarea`
Código de acceso a datos	`cargar()` y `guardar()` escritos en el DAO	Una clase base genérica `JsonDAOBase<T>` compartida
Creación de los DAO	`new` en el programa principal	`DAOFactory`, creada una sola

Aspecto	Parte I: consola	Parte II: aplicación móvil
		vez en la clase `Application`
Lectura y escritura de archivos	`java.nio.file`	`java.io` (válido en todas las versiones de Android del proyecto)
Reglas de negocio	Ninguna	Integridad entre `Materia` y `Tarea` y validaciones
Forma de trabajo	Individual	Equipo de 3 con roles, Git y revisión cruzada

La numeración de los pasos es continua para poder remitir de una parte a otra: **Pasos 1 a 6** en la Parte I y **Pasos 7 a 15** en la Parte II.

4. Idea general

El flujo es el mismo con una entidad o con varias: el cliente depende solo de la **interfaz** y lo único que cambia es la clase que lee y escribe.



Resaltado: el contrato, lo único que conoce el cliente. Punteado: almacén alternativo.

flujo del patrón DAO · Parte I y Parte II

En la Parte I el cliente es el `Main`, que habla con `EstudianteDAO`. En la Parte II los clientes son las Activities, que piden `MateriaDAO` y `TareaDAO` a la fábrica. En ningún caso el cliente menciona JSON ni archivos.

Parte I · DAO con archivo JSON en consola (Pasos 1 a 6)

En esta parte se trabaja de forma individual (unas 2 horas). Se construye un CRUD de una sola entidad, `Estudiante`, con un DAO que lee y escribe un archivo JSON. El programa principal depende únicamente de la **interfaz**; esa es la clave del patrón.

Estructura del proyecto

```
dao-json/
├── pom.xml
├── datos/
│   └── estudiantes.json
└── src/main/java/sv/edu/ufg/dao/
    ├── modelo/
    │   └── Estudiante.java
    ├── dao/
    │   ├── EstudianteDAO.java
    │   └── EstudianteDAOJson.java
    └── Main.java
```

Paso 1. Crear el proyecto y agregar Gson

Crea un proyecto Maven y agrega la dependencia en el `pom.xml`:

```
<dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.11.0</version>
</dependency>
```

Crea la carpeta `datos/` en la raíz del proyecto (al lado del `pom.xml`).

Paso 2. Crear la entidad

La entidad representa un registro. Es una clase simple con atributos, constructores, getters y setters. El constructor vacío es necesario para que Gson pueda reconstruir los objetos al leer el archivo.

```
package sv.edu.ufg.dao.modelo;

public class Estudiante {
    private int id;
    private String nombre;
    private String carrera;

    public Estudiante() { }

    public Estudiante(int id, String nombre, String carrera) {
        this.id = id;
        this.nombre = nombre;
        this.carrera = carrera;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
```

```
    public String getCarrera() { return carrera; }
    public void setCarrera(String carrera) { this.carrera = carrera; }

    @Override
    public String toString() {
        return id + " | " + nombre + " | " + carrera;
    }
}
```

Paso 3. Definir la interfaz DAO (el contrato)

```
package sv.edu.ufg.dao.dao;

import java.util.List;
import sv.edu.ufg.dao.modelo.Estudiante;

public interface EstudianteDAO {
    void insertar(Estudiante e);
    Estudiante buscar(int id);
    List<Estudiante> listar();
    boolean actualizar(Estudiante e);
    boolean eliminar(int id);
}
```

Idea clave: la interfaz no menciona JSON, archivos ni SQL. Solo declara qué se puede hacer con los datos, no cómo.

Paso 4. Implementar el DAO con JSON

La implementación carga todos los registros en una lista, los modifica en memoria y vuelve a escribir el archivo cuando hay cambios.

```
package sv.edu.ufg.dao.dao;

import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import com.google.gson.reflect.TypeToken;
import java.io.IOException;
import java.lang.reflect.Type;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import sv.edu.ufg.dao.modelo.Estudiante;

public class EstudianteDAOJson implements EstudianteDAO {

    private final Path archivo;
    private final Gson gson = new GsonBuilder().setPrettyPrinting().create();
    private final Type tipoLista = new TypeToken<List<Estudiante>>() { }.getType();

    public EstudianteDAOJson(String rutaArchivo) {
        this.archivo = Path.of(rutaArchivo);
    }
```

```
    // ---------- Operaciones del contrato ----------

    @Override
    public void insertar(Estudiante e) {
        List<Estudiante> lista = cargar();
        e.setId(siguienteId(lista));
        lista.add(e);
        guardar(lista);
    }

    @Override
    public Estudiante buscar(int id) {
        for (Estudiante e : cargar()) {
            if (e.getId() == id) {
                return e;
            }
        }
        return null;
    }

    @Override
    public List<Estudiante> listar() {
        return cargar();
    }

    @Override
    public boolean actualizar(Estudiante nuevo) {
        List<Estudiante> lista = cargar();
        for (int i = 0; i < lista.size(); i++) {
            if (lista.get(i).getId() == nuevo.getId()) {
                lista.set(i, nuevo);
                guardar(lista);
                return true;
            }
        }
        return false;
    }

    @Override
    public boolean eliminar(int id) {
        List<Estudiante> lista = cargar();
        boolean quitado = lista.removeIf(e -> e.getId() == id);
        if (quitado) {
            guardar(lista);
        }
        return quitado;
    }

    // ---------- Métodos privados reutilizables ----------

    private List<Estudiante> cargar() {
        try {
            if (Files.notExists(archivo)) {
                return new ArrayList<>();
            }
```

```
            String json = Files.readString(archivo, StandardCharsets.UTF_8);
            List<Estudiante> lista = gson.fromJson(json, tipoLista);
            return (lista != null) ? lista : new ArrayList<>();
        } catch (IOException ex) {
            throw new RuntimeException("No se pudo leer " + archivo, ex);
        }
    }

    private void guardar(List<Estudiante> lista) {
        try {
            Files.createDirectories(archivo.getParent());
            Files.writeString(archivo, gson.toJson(lista, tipoLista),
StandardCharsets.UTF_8);
        } catch (IOException ex) {
            throw new RuntimeException("No se pudo escribir " + archivo, ex);
        }
    }

    private int siguienteId(List<Estudiante> lista) {
        int mayor = 0;
        for (Estudiante e : lista) {
            mayor = Math.max(mayor, e.getId());
        }
        return mayor + 1;
    }
}
```

Observa la reutilización de código: los cinco métodos públicos comparten `cargar()`, `guardar()` y `siguienteId()`. Nadie repite la lectura ni la escritura del archivo. En la Parte II ese trío se saca de la clase y se convierte en una base genérica.

Paso 5. Probar desde el programa principal

Observa que la variable es del tipo **interfaz**, no de la clase concreta.

```
package sv.edu.ufg.dao;

import sv.edu.ufg.dao.dao.EstudianteDAO;
import sv.edu.ufg.dao.dao.EstudianteDAOJson;
import sv.edu.ufg.dao.modelo.Estudiante;

public class Main {
    public static void main(String[] args) {
        EstudianteDAO dao = new EstudianteDAOJson("datos/estudiantes.json");

        dao.insertar(new Estudiante(0, "Ana Martínez", "Ingeniería en Sistemas"));
        dao.insertar(new Estudiante(0, "Luis Pérez", "Ingeniería en Sistemas"));

        System.out.println("--- Listado ---");
        dao.listar().forEach(System.out::println);

        Estudiante e = dao.buscar(2);
        if (e != null) {
            e.setCarrera("Ingeniería Industrial");
            dao.actualizar(e);
```

```
        }

        dao.eliminar(1);

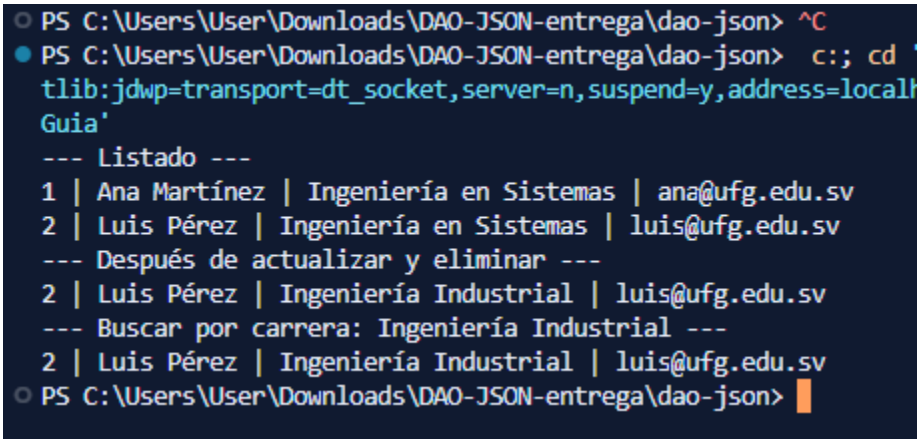
        System.out.println("--- Después de actualizar y eliminar ---");
        dao.listar().forEach(System.out::println);
    }
}
```

Paso 6. Verificar el archivo JSON

Después de ejecutar, abre `datos/estudiantes.json`. Debe verse así (el contenido depende de lo que hayas insertado, actualizado y eliminado):

```
[
  {
    "id": 2,
    "nombre": "Luis Pérez",
    "carrera": "Ingeniería Industrial"
  }
]
```

Si ejecutas el programa varias veces, los datos **persisten** entre ejecuciones porque están en el archivo.



Parte II · De la consola a una aplicación Android en equipo (Pasos 7 a 15)

En esta parte (unas 4 horas, en 2 sesiones de 2 horas) se construye una **aplicación móvil Android** con el mismo patrón, ahora con **dos entidades relacionadas**, una **clase base genérica** que evita repetir el código de lectura y escritura, una **fábrica de DAO** y **pantallas táctiles** que consumen únicamente las interfaces. El trabajo es en **equipo de 3**: el patrón permite que cada integrante avance en paralelo porque todos acuerdan primero el contrato.

Caso de estudio: agenda académica

La aplicación permite registrar las **materias** que cursa un estudiante y las **tareas** de cada una. Una materia tiene muchas tareas (relación de uno a muchos): cada tarea guarda el id de su materia en el campo `materiaId`.

Entidad	Atributos

Entidad	Atributos
`Materia`	`id`, `nombre`, `docente`
`Tarea`	`id`, `materiaId`, `titulo`, `fechaEntrega` (texto con formato aaaa-mm-dd), `completada`

Reglas que el equipo debe cumplir:

1. El nombre de la materia y el título de la tarea son obligatorios.
2. Una tarea solo se puede crear para una materia que existe.
3. No se puede eliminar una materia que todavía tiene tareas.
4. La fecha de entrega debe ser una fecha válida con el formato aaaa-mm-dd.

Organización del equipo

Cada equipo tiene tres integrantes. Los roles permiten trabajar en paralelo porque los tres parten del mismo contrato (las interfaces del Paso 8). También sirven para la revisión de código: cada integrante revisa el trabajo de otro.

Rol	Responsabilidad	Pasos
A. Modelo y contrato	Entidades e interfaces DAO; `DAOFactory` y clase `Application`; reglas de integridad	8 y 11
B. Persistencia	`JsonDAOBase<T>` y las implementaciones JSON; verificación de los archivos en el dispositivo	9, 10 y 14
C. Interfaz móvil	Layouts, adapter y Activities; validaciones de los formularios	12 y 13

Acuerdos de trabajo:

- **Contrato primero:** A sube al repositorio las entidades y las interfaces al comenzar la sesión; B y C programan contra ellas sin esperarse.
- **Una rama por rol:** cada integrante trabaja en su rama (`rol-a`, `rol-b`, `rol-c`) e integra a `main` con un pull request.
- **Revisión cruzada:** A revisa el código de B, B el de C y C el de A antes de aceptar cada pull request.
- **Commits propios:** cada integrante hace commits con su nombre; el historial es evidencia de su participación.
- **Cambios al contrato se avisan:** si una interfaz o una entidad debe cambiar, se informa al equipo antes, porque afecta el trabajo de los otros dos.

Estructura del proyecto Android

```
AgendaAcademica/
└── app/
    ├── build.gradle(.kts)
    └── src/main/
        ├── AndroidManifest.xml
        ├── java/sv/edu/ufg/agenda/
        │   ├── AgendaApp.java
        │   ├── modelo/
        │   │   ├── Entidad.java
        │   │   ├── Materia.java
        │   │   └── Tarea.java
        │   ├── dao/
        │   │   ├── DAO.java
        │   │   ├── MateriaDAO.java
        │   │   ├── TareaDAO.java
        │   │   ├── JsonDAOBase.java
        │   │   ├── MateriaDAOJson.java
        │   │   ├── TareaDAOJson.java
        │   │   └── DAOFactory.java
        │   └── ui/
        │       ├── MateriasActivity.java
        │       ├── MateriaAdapter.java
        │       └── TareasActivity.java
        └── res/layout/
            ├── activity_materias.xml
            ├── item_materia.xml
            └── activity_tareas.xml
```

Paso 7. Crear el proyecto Android y agregar dependencias

Un integrante crea el proyecto en Android Studio con la plantilla *Empty Views Activity*, lenguaje Java, paquete `sv.edu.ufg.agenda` y minSdk 24. Antes de subirlo al repositorio, agrega en el bloque `dependencies` de `app/build.gradle.kts` las siguientes líneas y presiona *Sync Now*. Los otros dos integrantes clonan el repositorio y crean su rama.

```
implementation("com.google.code.gson:gson:2.11.0")
implementation("androidx.recyclerview:recyclerview:1.3.2")
```

Si el archivo es `build.gradle` (Groovy), la sintaxis es con comillas simples y sin paréntesis: `implementation 'com.google.code.gson:gson:2.11.0'`.

Paso 8. Crear las entidades y el contrato (rol A)

Primero se define lo que los tres necesitan. Todas las entidades implementan `Entidad` para que la clase base genérica pueda leer y asignar el id sin conocer el tipo concreto.

```
package sv.edu.ufg.agenda.modelo;

public interface Entidad {
    int getId();
    void setId(int id);
}
```

Las entidades son clases simples con atributos, constructor vacío (lo necesita Gson), getters y setters. Compáralas con `Estudiante` del Paso 2: la única novedad es `implements Entidad`.

```
package sv.edu.ufg.agenda.modelo;

public class Materia implements Entidad {
    private int id;
    private String nombre;
    private String docente;

    public Materia() { }

    public Materia(int id, String nombre, String docente) {
        this.id = id;
        this.nombre = nombre;
        this.docente = docente;
    }

    @Override public int getId() { return id; }
    @Override public void setId(int id) { this.id = id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getDocente() { return docente; }
    public void setDocente(String docente) { this.docente = docente; }
}
```

```
package sv.edu.ufg.agenda.modelo;

public class Tarea implements Entidad {
    private int id;
    private int materiaId;
    private String titulo;
    private String fechaEntrega;   // texto con formato aaaa-mm-dd
    private boolean completada;

    public Tarea() { }

    public Tarea(int id, int materiaId, String titulo, String fechaEntrega) {
        this.id = id;
        this.materiaId = materiaId;
        this.titulo = titulo;
        this.fechaEntrega = fechaEntrega;
        this.completada = false;
    }

    @Override public int getId() { return id; }
    @Override public void setId(int id) { this.id = id; }
    public int getMateriaId() { return materiaId; }
    public void setMateriaId(int materiaId) { this.materiaId = materiaId; }
    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }
    public String getFechaEntrega() { return fechaEntrega; }
    public void setFechaEntrega(String fechaEntrega) { this.fechaEntrega =
fechaEntrega; }
```

```
    public boolean isCompletada() { return completada; }
    public void setCompletada(boolean completada) { this.completada = completada; }
}
```

Ahora el contrato. La interfaz genérica `DAO<T>` declara las cinco operaciones comunes (las mismas de `EstudianteDAO` en el Paso 3) y cada DAO específico agrega lo propio de su entidad:

```
package sv.edu.ufg.agenda.dao;

import java.util.List;

public interface DAO<T> {
    void insertar(T entidad);
    T buscar(int id);
    List<T> listar();
    boolean actualizar(T entidad);
    boolean eliminar(int id);
}
```

```
package sv.edu.ufg.agenda.dao;

import java.util.List;
import sv.edu.ufg.agenda.modelo.Materia;

public interface MateriaDAO extends DAO<Materia> {
    List<Materia> buscarPorNombre(String texto);
}
```

```
package sv.edu.ufg.agenda.dao;

import java.util.List;
import sv.edu.ufg.agenda.modelo.Tarea;

public interface TareaDAO extends DAO<Tarea> {
    List<Tarea> listarPorMateria(int materiaId);
}
```

Idea clave: estas interfaces son el contrato del equipo. B las implementa y C las consume; mientras no cambien, nadie bloquea a nadie. Ninguna menciona JSON, archivos ni Android.

Paso 9. Crear la clase base genérica `JsonDAOBase<T>` (rol B)

Aquí está la reutilización de código llevada un paso más allá que en la Parte I: `cargar()`, `guardar()` y `siguienteId()` se escriben **una sola vez** para cualquier entidad, en lugar de repetirse en cada DAO. En Android se usa `java.io` en lugar de `java.nio.file` porque funciona en todas las versiones de Android que admite el proyecto.

```
package sv.edu.ufg.agenda.dao;

import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import com.google.gson.reflect.TypeToken;
import java.io.File;
import java.io.FileInputStream;
```

```
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.Reader;
import java.io.Writer;
import java.lang.reflect.Type;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.List;
import sv.edu.ufg.agenda.modelo.Entidad;

public abstract class JsonDAOBase<T extends Entidad> implements DAO<T> {

    private final File archivo;
    private final Gson gson = new GsonBuilder().setPrettyPrinting().create();
    private final Type tipoLista;

    protected JsonDAOBase(File archivo, Class<T> clase) {
        this.archivo = archivo;
        this.tipoLista = TypeToken.getParameterized(List.class, clase).getType();
    }

    // ---------- Operaciones del contrato (iguales para cualquier entidad) ---------
-

    @Override
    public void insertar(T entidad) {
        List<T> lista = cargar();
        entidad.setId(siguienteId(lista));
        lista.add(entidad);
        guardar(lista);
    }

    @Override
    public T buscar(int id) {
        for (T e : cargar()) {
            if (e.getId() == id) {
                return e;
            }
        }
        return null;
    }

    @Override
    public List<T> listar() {
        return cargar();
    }

    @Override
    public boolean actualizar(T nuevo) {
        List<T> lista = cargar();
        for (int i = 0; i < lista.size(); i++) {
            if (lista.get(i).getId() == nuevo.getId()) {
                lista.set(i, nuevo);
```

```
                guardar(lista);
                return true;
            }
        }
        return false;
    }

    @Override
    public boolean eliminar(int id) {
        List<T> lista = cargar();
        boolean quitado = lista.removeIf(e -> e.getId() == id);
        if (quitado) {
            guardar(lista);
        }
        return quitado;
    }

    // ---------- Métodos reutilizables para las clases hijas ----------

    protected List<T> cargar() {
        if (!archivo.exists()) {
            return new ArrayList<>();
        }
        try (Reader lector = new InputStreamReader(
                new FileInputStream(archivo), StandardCharsets.UTF_8)) {
            List<T> lista = gson.fromJson(lector, tipoLista);
            return (lista != null) ? lista : new ArrayList<>();
        } catch (IOException ex) {
            throw new RuntimeException("No se pudo leer " + archivo, ex);
        }
    }

    protected void guardar(List<T> lista) {
        File carpeta = archivo.getParentFile();
        if (carpeta != null && !carpeta.exists() && !carpeta.mkdirs()) {
            throw new RuntimeException("No se pudo crear " + carpeta);
        }
        try (Writer escritor = new OutputStreamWriter(
                new FileOutputStream(archivo), StandardCharsets.UTF_8)) {
            gson.toJson(lista, tipoLista, escritor);
        } catch (IOException ex) {
            throw new RuntimeException("No se pudo escribir " + archivo, ex);
        }
    }

    private int siguienteId(List<T> lista) {
        int mayor = 0;
        for (T e : lista) {
            mayor = Math.max(mayor, e.getId());
        }
        return mayor + 1;
    }
}
```

Observa: el constructor recibe un `File`, no un `Context`. Así la capa de datos no depende de Android y se puede probar con JUnit sin emulador (ver la actividad de prueba unitaria).

Nota: en una aplicación de producción estas operaciones de archivo se ejecutan fuera del hilo principal. En esta práctica los archivos son pequeños y se acceden directamente para no añadir más complejidad.

Paso 10. Implementar los DAO con JSON (rol B)

Cada DAO concreto hereda el CRUD completo de la clase base y solo agrega el método propio de su entidad:

```
package sv.edu.ufg.agenda.dao;

import java.io.File;
import java.util.ArrayList;
import java.util.List;
import sv.edu.ufg.agenda.modelo.Materia;

public class MateriaDAOJson extends JsonDAOBase<Materia> implements MateriaDAO {

    public MateriaDAOJson(File archivo) {
        super(archivo, Materia.class);
    }

    @Override
    public List<Materia> buscarPorNombre(String texto) {
        List<Materia> resultado = new ArrayList<>();
        for (Materia m : cargar()) {
            if (m.getNombre().toLowerCase().contains(texto.toLowerCase())) {
                resultado.add(m);
            }
        }
        return resultado;
    }
}
```

```
package sv.edu.ufg.agenda.dao;

import java.io.File;
import java.util.ArrayList;
import java.util.List;
import sv.edu.ufg.agenda.modelo.Tarea;

public class TareaDAOJson extends JsonDAOBase<Tarea> implements TareaDAO {

    public TareaDAOJson(File archivo) {
        super(archivo, Tarea.class);
    }

    @Override
    public List<Tarea> listarPorMateria(int materiaId) {
        List<Tarea> resultado = new ArrayList<>();
        for (Tarea t : cargar()) {
```

```
            if (t.getMateriaId() == materiaId) {
                resultado.add(t);
            }
        }
        return resultado;
    }
}
```

Compara con la Parte I: en el Paso 4 cada DAO repetía `cargar()` y `guardar()`. Aquí esas operaciones viven en `JsonDAOBase<T>` y las clases hijas quedan pequeñas.

Paso 11. Crear la fábrica y la clase Application (rol A)

En la Parte I el programa principal creaba el DAO con `new`. Aquí esa decisión se centraliza: la fábrica crea los DAO una sola vez y la clase `Application` la mantiene viva mientras la app esté abierta. Las pantallas piden los DAO a la fábrica y nunca usan `new` con una clase JSON.

```
package sv.edu.ufg.agenda.dao;

import java.io.File;

public class DAOFactory {

    private final MateriaDAO materiaDAO;
    private final TareaDAO tareaDAO;

    public DAOFactory(File carpetaDatos) {
        this.materiaDAO = new MateriaDAOJson(new File(carpetaDatos,
"materias.json"));
        this.tareaDAO = new TareaDAOJson(new File(carpetaDatos, "tareas.json"));
    }

    public MateriaDAO materias() {
        return materiaDAO;
    }

    public TareaDAO tareas() {
        return tareaDAO;
    }
}
```

```
package sv.edu.ufg.agenda;

import android.app.Application;
import java.io.File;
import sv.edu.ufg.agenda.dao.DAOFactory;

public class AgendaApp extends Application {

    private DAOFactory daos;

    @Override
    public void onCreate() {
        super.onCreate();
```

```
        daos = new DAOFactory(new File(getFilesDir(), "datos"));
    }

    public DAOFactory daos() {
        return daos;
    }
}
```

Registra la clase en `AndroidManifest.xml` con el atributo `android:name` y declara las dos pantallas:

```
<application
    android:name=".AgendaApp"
    android:label="@string/app_name"
    ... >

    <activity android:name=".ui.MateriasActivity" android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
    <activity android:name=".ui.TareasActivity" />
</application>
```

Paso 12. Crear la pantalla de materias (rol C)

Renombra `MainActivity` a `MateriasActivity` y muévela al paquete `ui` (clic derecho > *Refactor*). Crea también una `TareasActivity` vacía (*File > New > Activity*) para que el proyecto compile; se completa en el Paso 13. Los textos se escriben directamente en los layouts para avanzar más rápido; en una app real irían en `strings.xml`.

`res/layout/activity_materias.xml`: formulario arriba y lista debajo.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <EditText
        android:id="@+id/etNombre"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Nombre de la materia"
        android:inputType="textCapWords" />

    <EditText
        android:id="@+id/etDocente"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Docente"
        android:inputType="textPersonName" />

    <Button
        android:id="@+id/btnGuardar"
```

```
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Agregar materia" />

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rvMaterias"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_marginTop="12dp"
        android:layout_weight="1" />
</LinearLayout>
```

res/layout/item_materia.xml: una fila de la lista, con el texto de la materia y dos botones.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="center_vertical"
    android:orientation="horizontal"
    android:paddingTop="8dp"
    android:paddingBottom="8dp">

    <TextView
        android:id="@+id/tvTexto"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textSize="16sp" />

    <Button
        android:id="@+id/btnTareas"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Tareas" />

    <Button
        android:id="@+id/btnEliminar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Eliminar" />
</LinearLayout>
```

MateriaAdapter.java: conecta la lista de materias con la fila. Avisa a la Activity qué botón se pulsó mediante una interfaz, sin conocer los DAO.

```
package sv.edu.ufg.agenda.ui;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.TextView;
import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;
import java.util.ArrayList;
import java.util.List;
```

```
import sv.edu.ufg.agenda.R;
import sv.edu.ufg.agenda.modelo.Materia;

public class MateriaAdapter extends RecyclerView.Adapter<MateriaAdapter.Fila> {

    public interface Acciones {
        void editar(Materia materia);
        void verTareas(Materia materia);
        void eliminar(Materia materia);
    }

    private final List<Materia> datos = new ArrayList<>();
    private final Acciones acciones;

    public MateriaAdapter(Acciones acciones) {
        this.acciones = acciones;
    }

    public void mostrar(List<Materia> nuevos) {
        datos.clear();
        datos.addAll(nuevos);
        notifyDataSetChanged();
    }

    @NonNull
    @Override
    public Fila onCreateViewHolder(@NonNull ViewGroup padre, int tipo) {
        View vista = LayoutInflater.from(padre.getContext())
                .inflate(R.layout.item_materia, padre, false);
        return new Fila(vista);
    }

    @Override
    public void onBindViewHolder(@NonNull Fila fila, int posicion) {
        Materia m = datos.get(posicion);
        fila.tvTexto.setText(m.getNombre() + "\n" + m.getDocente());
        fila.itemView.setOnClickListener(v -> acciones.editar(m));
        fila.btnTareas.setOnClickListener(v -> acciones.verTareas(m));
        fila.btnEliminar.setOnClickListener(v -> acciones.eliminar(m));
    }

    @Override
    public int getItemCount() {
        return datos.size();
    }

    static class Fila extends RecyclerView.ViewHolder {
        final TextView tvTexto;
        final Button btnTareas;
        final Button btnEliminar;

        Fila(@NonNull View vista) {
            super(vista);
            tvTexto = vista.findViewById(R.id.tvTexto);
            btnTareas = vista.findViewById(R.id.btnTareas);
```

```
            btnEliminar = vista.findViewById(R.id.btnEliminar);
        }
    }
}
```

`MateriasActivity.java`: usa solo las **interfaces** `MateriaDAO` y `TareaDAO`. Un toque sobre la fila carga la materia en el formulario para editarla.

```
package sv.edu.ufg.agenda.ui;

import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;
import sv.edu.ufg.agenda.AgendaApp;
import sv.edu.ufg.agenda.R;
import sv.edu.ufg.agenda.dao.DAOFactory;
import sv.edu.ufg.agenda.dao.MateriaDAO;
import sv.edu.ufg.agenda.dao.TareaDAO;
import sv.edu.ufg.agenda.modelo.Materia;

public class MateriasActivity extends AppCompatActivity implements
MateriaAdapter.Acciones {

    private MateriaDAO materiaDAO;
    private TareaDAO tareaDAO;
    private MateriaAdapter adapter;
    private EditText etNombre;
    private EditText etDocente;
    private Button btnGuardar;
    private Materia enEdicion;   // null = se está agregando una materia nueva

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_materias);

        DAOFactory daos = ((AgendaApp) getApplication()).daos();
        materiaDAO = daos.materias();
        tareaDAO = daos.tareas();

        etNombre = findViewById(R.id.etNombre);
        etDocente = findViewById(R.id.etDocente);
        btnGuardar = findViewById(R.id.btnGuardar);

        adapter = new MateriaAdapter(this);
        RecyclerView lista = findViewById(R.id.rvMaterias);
        lista.setLayoutManager(new LinearLayoutManager(this));
        lista.setAdapter(adapter);

        btnGuardar.setOnClickListener(v -> guardar());
        refrescar();
```

```
    }

    private void guardar() {
        String nombre = etNombre.getText().toString().trim();
        String docente = etDocente.getText().toString().trim();
        if (nombre.isEmpty()) {
            etNombre.setError("El nombre es obligatorio");
            return;
        }
        if (enEdicion == null) {
            materiaDAO.insertar(new Materia(0, nombre, docente));
        } else {
            enEdicion.setNombre(nombre);
            enEdicion.setDocente(docente);
            materiaDAO.actualizar(enEdicion);
        }
        limpiarFormulario();
        refrescar();
    }

    private void limpiarFormulario() {
        enEdicion = null;
        etNombre.setText("");
        etDocente.setText("");
        btnGuardar.setText("Agregar materia");
    }

    private void refrescar() {
        adapter.mostrar(materiaDAO.listar());
    }

    @Override
    public void editar(Materia materia) {
        enEdicion = materia;
        etNombre.setText(materia.getNombre());
        etDocente.setText(materia.getDocente());
        btnGuardar.setText("Actualizar materia");
    }

    @Override
    public void verTareas(Materia materia) {
        Intent intento = new Intent(this, TareasActivity.class);
        intento.putExtra("materiaId", materia.getId());
        startActivity(intento);
    }

    @Override
    public void eliminar(Materia materia) {
        if (!tareaDAO.listarPorMateria(materia.getId()).isEmpty()) {
            Toast.makeText(this, "No se puede eliminar: la materia tiene tareas",
                    Toast.LENGTH_LONG).show();
            return;
        }
        materiaDAO.eliminar(materia.getId());
        if (enEdicion != null && enEdicion.getId() == materia.getId()) {
```

```
            limpiarFormulario();
        }
        refrescar();
    }
}
```

Idea clave: `MateriasActivity` solo importa la fábrica y las interfaces. Si la clase `MateriaDAOJson` desapareciera mañana, esta pantalla no se enteraría.

Paso 13. Crear la pantalla de tareas (todo el equipo)

Esta pantalla se construye con el mismo patrón que la anterior y es el reto de integración del equipo: C diseña la pantalla, B aporta los métodos de `TareaDAO` que hagan falta y A verifica que se cumplan las reglas de integridad.

Requisito	Comportamiento esperado
Entrada	Recibe `materiaId` por *Intent*; si `materiaDAO.buscar(materiaId)` devuelve `null`, cierra la pantalla
Encabezado	Muestra el nombre de la materia y cuántas tareas están completadas, por ejemplo «2 de 5 completadas»
Lista	Muestra solo las tareas de esa materia con `listarPorMateria()`, ordenadas por fecha de entrega
Formulario	Título y fecha (aaaa-mm-dd), con las validaciones de las reglas 1, 2 y 4
Acciones	Agregar, editar, eliminar y marcar una tarea como completada con un `CheckBox`
Persistencia	Cada cambio se refleja de inmediato en `tareas.json`

Pistas para el código. Recibir la materia y ordenar la lista (el formato aaaa-mm-dd se ordena como texto igual que como fecha):

```
// Al iniciar la pantalla
int materiaId = getIntent().getIntExtra("materiaId", -1);
Materia materia = materiaDAO.buscar(materiaId);
if (materia == null) {
    finish();
    return;
}

// Al refrescar la lista
List<Tarea> tareas = tareaDAO.listarPorMateria(materiaId);
Collections.sort(tareas, (a, b) ->
a.getFechaEntrega().compareTo(b.getFechaEntrega()));
```

Validar que el texto sea una fecha real (rechaza, por ejemplo, 2026-02-30):

```
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Locale;

private boolean fechaValida(String texto) {
    SimpleDateFormat formato = new SimpleDateFormat("yyyy-MM-dd", Locale.US);
    formato.setLenient(false);
    try {
        formato.parse(texto);
        return true;
    } catch (ParseException ex) {
        return false;
    }
}
```

Paso 14. Verificar los archivos JSON en el dispositivo (rol B)

En la Parte I el JSON estaba en `datos/`, dentro del proyecto. En Android vive en el almacenamiento interno de la app. Con la app ejecutándose en el emulador, abre *Device Explorer* (*View > Tool Windows*) y navega a `/data/data/sv.edu.ufg.agenda/files/datos/`. Allí están `materias.json` y `tareas.json`. Después de registrar una materia y una tarea deben verse así (el contenido depende de tus datos):

```
[
  {
    "id": 1,
    "nombre": "Programación II",
    "docente": "Ing. Pérez"
  }
]
```

```
[
  {
    "id": 1,
    "materiaId": 1,
    "titulo": "Guía práctica 3",
    "fechaEntrega": "2026-10-20",
    "completada": false
  }
]
```

Cierra la app por completo y vuelve a abrirla: los datos **persisten** porque están en los archivos. Si desinstalas la app, el almacenamiento interno se borra junto con ella.





Paso 15. Integrar y probar en equipo

Antes de entregar, el equipo integra todo en la rama `main` y la prueba desde cero:

- Cada integrante abre su pull request y otro lo revisa (A revisa a B, B a C y C a A); se aprueba o se piden cambios.
- Con `main` actualizada, ejecuten la app en un emulador limpio y recorran el flujo completo: crear una materia, agregar tareas, editar, marcar una como completada e intentar eliminar una materia que tiene tareas (debe aparecer el mensaje).
- Cierren la app, ábranla de nuevo y confirmen que los datos siguen ahí.
- Revisen que ninguna pantalla importe `MateriaDAOJson` ni `TareaDAOJson`.

5. Cambiar de JSON a base de datos sin tocar al cliente

La gran ventaja del patrón es la misma en las dos partes: el cliente (el `Main` o las Activities) solo conoce las interfaces. Para pasar a una base de datos basta con crear **otras clases** que cumplan los mismos contratos y cambiar el punto donde se construyen.

	Parte I (MySQL con JDBC)	Parte II (SQLite)
Se crea	`EstudianteDAOJdbc`	`MateriaDAOSqlite` y `TareaDAOSqlite`
Se modifica	Una línea del `Main`	Solo el constructor de `DAOFactory`
No se toca	Menús, validaciones e impresión	Activities, adapters y layouts

En la Parte I:

```
public class EstudianteDAOJdbc implements EstudianteDAO {
    // misma interfaz, pero con Connection, PreparedStatement y SQL
}
```

```
// Antes
EstudianteDAO dao = new EstudianteDAOJson("datos/estudiantes.json");
// Después
EstudianteDAO dao = new EstudianteDAOJdbc();
```

En la Parte II, la fábrica recibiría el `Context` en lugar de la carpeta:

```
public class MateriaDAOSqlite implements MateriaDAO {
    // mismas operaciones, pero con SQLiteDatabase (o Room) y SQL
}

public class TareaDAOSqlite implements TareaDAO {
    // mismas operaciones, pero con SQLiteDatabase (o Room) y SQL
}
```

```
// Antes
this.materiaDAO = new MateriaDAOJson(new File(carpetaDatos, "materias.json"));
// Después
this.materiaDAO = new MateriaDAOSqlite(contexto);
```

La fábrica de la Parte II ya hace lo que en la Parte I solo se sugería: **centralizar la creación del DAO** para que el cambio de almacén ocurra en un único lugar. Si quieres lograrlo también en la Parte I, crea una clase `DAOFactory` con un método `estudiantes()` y úsala desde el `Main`.

6. Actividades para el estudiante

Las actividades de la Parte I se resuelven de forma individual antes de la Parte II. El reto final de la Parte I anticipa la clase base de la Parte II: conviene intentarlo antes de leer el Paso 9 y comparar después las soluciones.

Parte I (individual)

- **Reproducir:** crea el proyecto completo y ejecuta el `Main`. Entrega captura del archivo `estudiantes.json` resultante.
- **Ampliar la entidad:** agrega el atributo `correo` a `Estudiante` y verifica que aparece en el JSON.
- **Nuevo método:** agrega `List<Estudiante> buscarPorCarrera(String carrera)` a la interfaz y a la implementación.
- **Menú por consola:** crea un menú (1. Insertar, 2. Listar, 3. Buscar, 4. Actualizar, 5. Eliminar, 0. Salir) que use solo la interfaz `EstudianteDAO`.
- **Segunda entidad:** crea `Curso` con su `CursoDAO` y `CursoDAOJson`, reutilizando la misma estrategia de `cargar()` y `guardar()`.
- **Reto:** elimina el código repetido entre `EstudianteDAOJson` y `CursoDAOJson` creando una clase genérica `JsonDAOBase<T>`.

Parte II (equipo)

- **Reproducir (equipo):** construyan el proyecto siguiendo los Pasos 7 al 12 y ejecútenlo en el emulador. Entreguen captura de `materias.json` en *Device Explorer*.
- **Completar (equipo):** desarrollen `TareasActivity` según el Paso 13 y entreguen captura de `tareas.json` con al menos tres tareas de dos materias distintas.

- **Ampliar la entidad (rol A):** agrega el atributo `prioridad` (Alta, Media o Baja) a `Tarea`, avisa al equipo del cambio y verifica que aparece en el JSON y en la pantalla.
- **Búsqueda (rol C con apoyo de B):** agrega un campo de búsqueda en `MateriasActivity` que use `buscarPorNombre()`, y agrega `List<Tarea> buscarPorTitulo(String texto)` a `TareaDAO` y a su implementación para usarlo en la pantalla de tareas.
- **Resumen por materia (equipo):** en la lista de materias muestra, bajo cada nombre, cuántas tareas pendientes tiene. Decidan en qué capa va el cálculo y justifiquen la decisión.
- **Reto, DAO en memoria (rol C):** crea `MateriaDAOMemoria` y `TareaDAOMemoria` que guarden los datos en una lista, y cambia una sola línea de `DAOFactory` para alternar entre memoria y JSON. Explica en qué situaciones del trabajo en equipo sirve.
- **Reto, prueba unitaria (rol B):** escribe una prueba JUnit para `TareaDAOJson` con `TemporaryFolder` que verifique insertar, listar por materia y eliminar, sin usar el emulador.

7. Errores frecuentes

Los cuatro primeros errores se repiten en las dos partes porque vienen de la misma lógica de archivo JSON; los demás son propios de cada plataforma.

Parte	Error	Causa probable	Solución
Ambas	Los datos salen vacíos o con campos nulos	Falta el constructor vacío, cambió el nombre de un atributo o el archivo quedó mal formado	Agrega `public Estudiante() { }` (o `public Tarea() { }`), revisa los nombres y borra el archivo para que se regenere
Ambas	`JsonSyntaxException`	El archivo JSON fue editado a mano y quedó mal formado	Valida el JSON o elimina el archivo para que se regenere
Ambas	Los IDs se repiten	Se asignó el ID manualmente	Deja que `siguienteId()` lo genere
Ambas	Los cambios no aparecen en el archivo	Se modificó la lista pero no se llamó a `guardar()`	Revisa que cada operación de escritura termine con `guardar(lista)`
I	`NoSuchFileException`	La carpeta `datos/` no existe o la ruta es incorrecta	El método `guardar()` crea la carpeta; revisa que la ruta sea relativa a la raíz del proyecto
II	`Cannot resolve symbol` con Gson o	Falta la dependencia o no se sincronizó Gradle	Agrega las líneas del Paso 7 y presiona *Sync*

Parte	Error	Causa probable	Solución
	RecyclerView		*Now*
II	La app se cierra al abrir con `ClassCastException`	`AgendaApp` no está registrada en el manifiesto	Agrega `android:name=".AgendaApp"` en la etiqueta `<application>`
II	La lista no se actualiza después de agregar o eliminar	No se volvió a cargar la lista tras la operación	Después de cada escritura llama a `refrescar()` para que el adapter reciba `listar()`
II	No aparece la carpeta `datos/` en *Device Explorer*	Aún no se ha guardado nada o la vista no se actualizó	Registra una materia y usa *Synchronize* sobre la carpeta `files`
II	Funciona en *debug* pero no en *release*	El minimizador renombra los atributos y Gson ya no coincide con el JSON	Para esta práctica ejecuta en *debug*; en producción se agregan reglas de conservación o `@SerializedName`
II	Conflictos al integrar las ramas	Dos integrantes editaron el mismo archivo	Respeten los roles, hagan `git pull` antes de `git push` y resuelvan el conflicto juntos

8. Criterios de evaluación

Los mismos criterios se aplican a las dos partes; cambia lo que se observa en cada una.

Criterio	Parte I (individual)	Parte II (equipo)
Separación de responsabilidades	Entidad, interfaz e implementación están en clases distintas y el programa usa la interfaz	Entidades, interfaces, implementaciones y pantallas están en paquetes distintos y las pantallas usan solo las interfaces
CRUD y reglas	Las cinco operaciones funcionan y el JSON refleja los cambios	Las operaciones de ambas entidades funcionan y se cumplen las cuatro reglas del caso de estudio

Criterio	Parte I (individual)	Parte II (equipo)
Reutilización de código	No hay lectura o escritura de archivo repetida; se usan métodos privados compartidos	`JsonDAOBase<T>` es usada por los dos DAO y no hay lectura ni escritura de archivo repetida
Aplicación móvil funcional	No aplica	La app corre en el emulador o en un celular y los datos persisten al cerrarla y abrirla
Trabajo en equipo	No aplica	Cada integrante cumplió su rol, hay commits de los tres y los pull requests fueron revisados
Calidad y orden	Nombres claros, paquetes ordenados, código legible	Nombres claros, paquetes ordenados, código legible
Diligencia (Saber ser)	Entrega completa, probada y con las actividades resueltas	Entrega completa, probada y con las actividades resueltas

9. Cierre

- El **DAO** separa el acceso a datos del resto de la aplicación, sea un programa de consola o las pantallas de una app móvil, y esa separación es la que permite que tres personas trabajen en paralelo.
- La **interfaz** es el contrato; la implementación (JSON, JDBC, SQLite u otra) es intercambiable.
- Una **clase base genérica** y una **fábrica** eliminan el código repetido y concentran en un solo lugar el cambio de almacén.
- Reutilizar código con patrones de diseño es una práctica profesional que se nota en la mantenibilidad del proyecto y en la fluidez del equipo.

Pregunta de reflexión: si mañana la institución exige guardar los datos en una base de datos, ¿cuántas clases tendrías que modificar en tu proyecto de consola y cuántos archivos de la carpeta `ui` en tu app móvil? Si la respuesta es «una» y «ninguno», aplicaste bien el patrón.

R// En el proyecto de consola habría que modificar una clase, DAOFactory, y crear las nuevas implementaciones, como EstudianteDAOJdbc. En la app móvil, ninguno de los archivos de la carpeta ui cambia: solo se crean MateriaDAOSqlite y TareaDAOSqlite y se modifica DAOFactory. Por eso se aplicó bien el patrón: las pantallas y el Main solo conocen las interfaces.

Link de repositorio: https://github.com/JaviRoak/agenda-academica